TOOLKIT · BACKEND ENGINEERING HANDBOOK

KAFKA COMPLETE HANDBOOK

Distributed Event Streaming, Partitioning, Delivery Semantics, Transactional Outbox & Production Incident Mastery

Phase	II — Backend Infrastructure
Tool ID	08 of 13
Curriculum Scope	Complete 3-Chunk Comprehensive Curriculum
Deliverable Type	Consolidated Tool Engineering Handbook (Release Build)
Source Authority	content/kafka/ (01, 02, and 03 markdown sources)
Target Audience	Backend Software Engineers, SDE-1 / SDE-2, Distributed Systems Architects
Engineering Focus	Event-Driven Architecture, Log Partitioning, Dual-Write Outbox Pattern, Consumer Lag Triage & High-Availability Clusters

This comprehensive handbook provides an authoritative, production-grade guide to **Apache Kafka** for modern backend engineering. Spanning three rigorous pedagogical layers—Mental Models, Practical Engineering, and Production Incident Mastery—it equips engineers to design resilient event-driven architectures, master partitioned log mechanics (topics, partitions, offsets, broker clusters), configure robust Spring Boot producers and consumers, implement bulletproof idempotency with relational deduplication, solve the database–event dual-write problem using the Transactional Outbox pattern, tune high-throughput pipelines, and diagnose production consumer lag, poison messages, and broker failovers.

Curriculum Structure & Core Topics

Part I — Fundamentals, Event Streaming & Core Mental Models: Distributed commit log principles, commands vs events, broker architecture, partition-scoped ordering, retention vs compaction, consumer group assignment, Docker Compose KRaft cluster setup, and basic CLI workflows.

Part II — Producers, Consumers, Delivery Semantics & Spring Boot Integration: Producer batching (batch.size, linger.ms), acknowledgment semantics (acks=all), idempotent producers, consumer fetch vs process loops, manual offset commits, Spring Kafka integration, relational deduplication, and the Transactional Outbox pattern.

Part III — Production Reliability, Security, Performance & Incident Mastery: Partition key design, hot partition mitigation, in-sync replicas (min.insync.replicas), consumer lag triage runbooks, dead-letter topics (DLT), safe replay protocols, TLS/SASL security, distributed tracing, and incident response simulations.

PART I: FUNDAMENTALS, EVENT STREAMING & CORE MENTAL MODELS

Chunk 1 of 3 · Backend Engineering Handbook

Kafka — Chunk 1: Fundamentals, Event Streaming & Core Mental Models

Backend Engineering Tools & Infrastructure Handbook (BEH)

Source file: `content/kafka/01-kafka-fundamentals-event-streaming-core-mental-models.md`

---

1. Learning Objectives

By the end of this chapter, you should be able to:

- Explain why Kafka exists and which problems it solves.
- Understand producers, brokers, topics, partitions, offsets, and consumers.
- Explain how Kafka differs from traditional message queues and Redis.
- Understand consumer groups and how Kafka distributes work.
- Create a local Kafka environment and perform basic operations.
- Recognize delivery semantics and common messaging mistakes.
- Explain Kafka's role in a production backend architecture.
- Answer foundational Kafka interview questions confidently.

This chapter begins the Kafka curriculum. Across all three planned Kafka chunks, we will cover fundamentals, practical engineering workflows, and production engineering mastery.

Layer 1 — Fundamentals & Mental Models

2. What Is Apache Kafka?

Apache Kafka is a **distributed event-streaming platform** used to publish, store, and consume streams of events.

Imagine an e-commerce application. When a customer places an order, several things may need to happen:

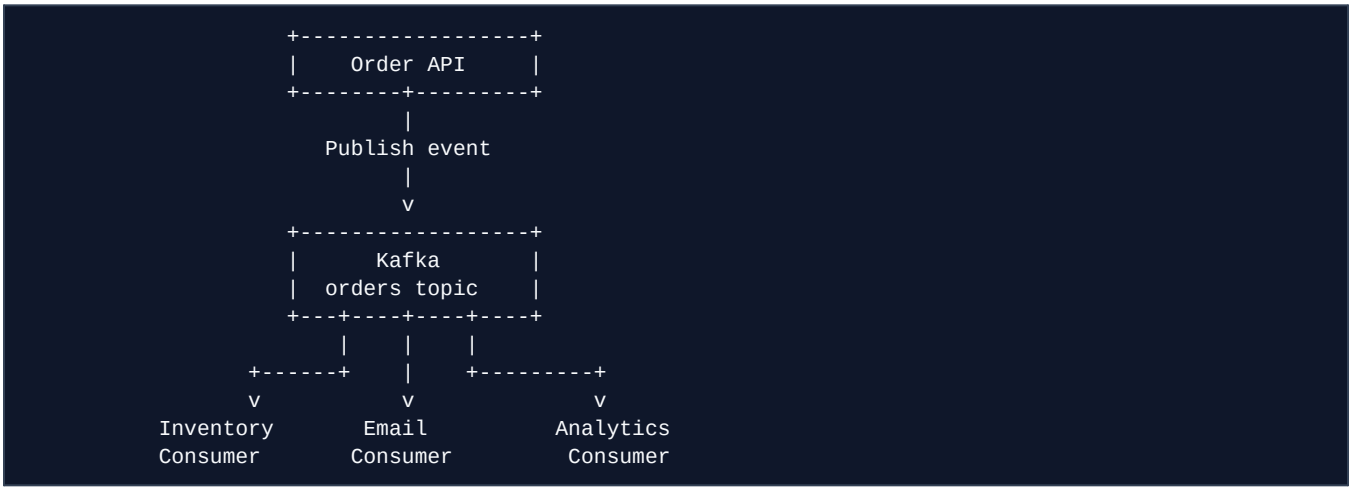
1. The order must be saved.
2. Inventory must be updated.
3. A confirmation email must be sent.
4. An analytics system must record the purchase.
5. A fraud-detection system may need to evaluate the transaction.

A naive implementation calls every downstream service directly from the order API.

```
Customer
   |
   v
Order API
   |
   +----> Inventory Service
   |
   +----> Email Service
   |
   +----> Analytics Service
   |
   +----> Fraud Service
```

This creates coupling. If the email service is slow or unavailable, it can delay the order request. Adding more downstream services also makes the order API increasingly complicated.

Kafka introduces an event stream between producers and consumers.



The order service publishes an event such as `OrderCreated`. Independent consumers process that event for their own purposes.

Kafka does not automatically make every downstream operation succeed. It provides a durable, distributed event log that helps services communicate asynchronously and independently.

The central mental model

Kafka is a distributed, replayable log of events.

This is more useful than thinking of Kafka as merely a pipe that moves messages from A to B.

A consumer can read events, track its position, and potentially read them again when it needs to recover or rebuild state, provided the events remain available under the configured retention policy.

3. What Is an Event?

An event records something that has happened.

Examples:

- `OrderCreated`
- `PaymentAuthorized`
- `InventoryReserved`
- `UserRegistered`
- `ShipmentDispatched`

A simplified event might look like this:

```
{
  "eventId": "evt-1001",
  "eventType": "OrderCreated",
  "orderId": "ord-782",
  "customerId": "cust-45",
  "totalAmount": 1499.00,
  "currency": "INR",
  "occurredAt": "2026-10-23T08:30:00Z"
}
```

This is an illustrative event payload, not a Kafka-specific wire format.

Useful event-design principles:

- Give events a clear name describing something that happened.
- Include identifiers needed by consumers.
- Include an event ID when consumers need to detect duplicate processing.
- Use a consistent timestamp and schema strategy.
- Avoid putting secrets or unnecessary personal information into events.
- Consider backward compatibility when changing event schemas.

Commands versus events

A command requests an action. An event records a fact.

Command	Event
`CreateOrder`	`OrderCreated`
`ChargePayment`	`PaymentCharged`
`ReserveInventory`	`InventoryReserved`

A command can be rejected. An event normally describes a fact that has already occurred.

For example, `ChargePayment` asks a payment service to charge a customer. `PaymentCharged` states that the charge succeeded.

This distinction matters when designing event-driven systems.

4. Kafka's Core Components

Producer

A **producer** publishes records to Kafka.

A producer may be an order service, payment service, background worker, or any other application that emits events.

A Kafka record can contain:

- A key
- A value
- A timestamp
- Headers
- Other record metadata

Example:

```
Key:   order-782
Value: OrderCreated event
Topic: orders
```

The key can influence partition selection. It is not simply decorative metadata.

Broker

A **broker** is a Kafka server that stores and serves topic partitions.

A Kafka cluster typically contains multiple brokers. Partitions can be distributed among them so the cluster can scale and tolerate certain failures.

A broker is not the same as a topic, partition, or consumer.

Topic

A **topic** is a named stream of records.

Examples:

```
orders
payments
inventory-events
notifications
```

Topics organize events by domain or purpose.

A topic is divided into partitions. It is not necessarily stored entirely on one broker.

Partition

A **partition** is an ordered, append-only log within a topic.

Imagine the `orders` topic has three partitions:

```
orders
  |
  +-- Partition 0: [E0] [E1] [E2] [E3]
  |
  +-- Partition 1: [E0] [E1] [E2]
  |
  +-- Partition 2: [E0] [E1] [E2] [E3] [E4]
```

The offsets shown inside different partitions are local to those partitions. They do not represent one global topic-wide sequence.

Kafka guarantees ordering **within a partition**, not across all partitions of a topic.

Partitioning is a major part of Kafka's scalability model. Different partitions can be processed concurrently, subject to consumer-group assignments and other constraints.

Offset

An **offset** is the position of a record within a partition.

For example:

```
Partition 0:
Offset 0 -> OrderCreated
Offset 1 -> OrderCreated
Offset 2 -> OrderCancelled
Offset 3 -> OrderCreated
```

A consumer tracks its progress through a partition using offsets.

Offsets are not globally unique across an entire topic. The meaningful record location is the combination of topic, partition, and offset.

Example:

```
orders / partition 0 / offset 42
```

A consumer can resume reading from a position after a restart if its progress has been committed and the required records remain available.

Consumer

A **consumer** reads records from Kafka and performs application work.

Examples:

- An inventory consumer updates stock.
- An email consumer schedules a confirmation.
- An analytics consumer aggregates sales.
- A fraud consumer evaluates suspicious activity.

Kafka does not automatically execute the business logic in your consumer. Your application must process the record, handle errors, and manage progress appropriately.

Consumer group

A **consumer group** is a set of consumers that cooperatively process a topic's partitions.

Within one consumer group, a partition is assigned to at most one consumer at a time under the normal group-assignment model.

Suppose a topic has four partitions.

```
Topic: orders

P0 ■■■■■> Consumer A
P1 ■■■■■> Consumer B
P2 ■■■■■> Consumer C
P3 ■■■■■> Consumer D
```

If the group has only two consumers, each may receive multiple partitions.

If the group has six consumers but only four partitions, at least two consumers will have no partition assigned for that topic at that time.

Key rule: For a single topic, adding consumers to one group beyond the number of partitions does not increase the number of consumers that can concurrently own those partitions.

5. How a Kafka Record Travels

Consider a customer placing an order.

1. The Order API constructs an `OrderCreated` record.
2. The producer sends it to the `orders` topic.
3. Kafka routes it to a partition.
4. The broker appends the record to that partition's log.
5. A consumer in a subscribed group fetches the record.
6. The consumer executes its business logic.
7. The consumer commits progress according to its processing strategy.

```
Application
    |
    v
Producer
    |
    v
Topic / Partition
    |
    v
Broker appends record
    |
    v
Consumer fetches record
    |
    v
Business processing
    |
    v
Offset progress committed
```

The last two steps are particularly important. **Reading a record and successfully completing its business operation are not the same thing.**

If a consumer commits its offset before completing the operation, a crash may cause the operation to be skipped after recovery. If it commits only after processing, a crash between the operation and the commit can cause the record to be processed again.

That is why Kafka consumers must be designed around retries, idempotency, and correct progress management.

6. Kafka's Ordering Model

Kafka's ordering guarantees are scoped to partitions.

Suppose order events use `orderId` as the key:

```
Order 100 -> partition 1
Order 100 -> partition 1
Order 100 -> partition 1

Order 200 -> partition 2
Order 200 -> partition 2
```

With a stable partitioning strategy and unchanged topic partitioning assumptions, records with the same key are generally routed to the same partition. That helps preserve their relative order.

However, do not treat this as an unconditional guarantee across all configurations or future partition-count changes. Partitioning strategies, custom partitioners, and changing the number of partitions can affect placement.

When ordering matters

Ordering is important when events describe a sequence of state transitions.

For example:

```
AccountOpened
FundsDeposited
FundsWithdrawn
AccountClosed
```

Processing these events out of order may produce incorrect results.

For a backend system, ask:

- What entity needs ordered processing?
- What key represents that entity?
- Can two records for the same entity be processed concurrently?
- What happens when a record fails?
- Is global ordering actually necessary?

Global ordering across a highly parallel topic is expensive and should not be assumed by default.

7. Kafka Versus Traditional Message Queues

Kafka and traditional queues can both support asynchronous communication, but their typical models differ.

Dimension	Kafka	Traditional work queue
Core model	Distributed event log	Messages delivered to workers
Consumption	Consumers track positions	Messages often acknowledged and removed from active delivery
Replay	Supported while records remain available	Depends on the queue and its configuration

Dimension	Kafka	Traditional work queue
Multiple independent consumers	Consumer groups can independently read a topic	Usually requires separate queues or fan-out configuration
Ordering	Per partition	Depends on queue type and delivery configuration
Scaling	Partitions and consumer groups	Queue and worker scaling mechanisms

These are general patterns, not universal rules. Modern queue systems offer features that overlap with Kafka, and Kafka's behavior depends on its configuration and application design.

Example: background email

If a user registers and the system needs to send one welcome email, a conventional task queue may be sufficient.

Example: order events shared by multiple services

If inventory, analytics, fraud detection, and reporting all need to consume order events independently, Kafka can be a natural fit because each consumer group can track its own progress.

Choose based on the delivery and replay requirements, operational complexity, ordering needs, and throughput—not on popularity alone.

8. Kafka Versus Redis

Kafka and Redis solve different primary problems, even though both can appear in distributed backend architectures.

Dimension	Kafka	Redis
Primary role	Distributed event streaming and durable log	In-memory data platform, often used for caching and fast data access
Typical data model	Ordered records within partitions	Keys, values, collections, and specialized structures
Common use	Event pipelines, asynchronous integration, replay	Caching, sessions, rate limiting, coordination
Replay	Read records again while retained	Depends on the Redis feature and data-retention design
Work distribution	Consumer groups and partition assignments	Depends on the mechanism used, such as Streams or other structures
Latency profile	Optimized for high-throughput event streaming	Often extremely low-latency access to in-memory data

Redis also supports Streams and consumer groups. It is therefore incorrect to claim Redis cannot perform stream processing or asynchronous messaging.

The real distinction is the required system behavior. Kafka is built around a distributed event log and partitioned streaming. Redis is a broader in-memory data platform frequently used for fast access, caching, and lightweight coordination.

Practical architecture

```
Order API
   |
   +----> PostgreSQL: durable business state
   |
   +----> Kafka: order events for downstream consumers
   |
   +----> Redis: cache frequently accessed data
```

This is a conceptual architecture, not a complete transaction-safe implementation. Writing to PostgreSQL and publishing to Kafka introduces a dual-write problem, which we will address in later engineering work.

9. Retention: Why Kafka Is Not Just a Queue

Kafka commonly retains records according to a configured retention policy.

A consumer reading a record does not ordinarily delete that record for every other consumer. Multiple consumer groups can read the same retained data independently.

For example:

```
orders topic
  |
  +----> inventory-group
  |
  +----> analytics-group
  |
  +----> fraud-group
```

All three groups can read the same stream independently and maintain their own positions.

Retention can be time-based, size-based, or configured using other topic policies. A record is not guaranteed to remain available forever. Compaction can also change which historical records are retained for compacted topics.

This gives Kafka two important characteristics:

- **Decoupling:** producers do not need to know every downstream consumer.
- **Replay:** consumers can reread retained records when appropriate.

Replay is useful for rebuilding projections, backfilling analytics, and recovering consumer state. It must be designed carefully when replayed events trigger external side effects such as sending emails or charging payments.

10. Delivery Semantics: The First Mental Model

Three terms appear frequently in Kafka discussions.

At-most-once

A record may be lost from the application's processing perspective, but duplicate processing is avoided by the chosen strategy.

One way this can happen is when progress is committed before business processing. If the application crashes after committing but before completing the operation, the record may be skipped after restart.

At-least-once

A record is processed one or more times. Duplicates are possible.

A common approach is to process the record and commit the offset after successful processing. If the operation succeeds but the application crashes before the offset commit, the record can be delivered again.

Exactly-once

Exactly-once semantics require careful definition of what is guaranteed and within which boundary.

Kafka provides transactional and idempotent-producer capabilities for specific Kafka operations. However, exactly-once effects across Kafka, PostgreSQL, payment providers, email systems, and arbitrary external services do not arise automatically.

A common production approach is:

1. Expect duplicate delivery where appropriate.
2. Give events stable IDs.
3. Make consumer-side state changes idempotent.
4. Use database constraints or deduplication records where appropriate.
5. Commit offsets in coordination with the processing design.
6. Use Kafka transactions or an outbox/inbox pattern when the problem calls for them.

We will explore reliability and transactional boundaries in the later Kafka chunks.

Layer 2 — Practical Engineering & Workflows

11. Set Up Kafka Locally

For learning, run Kafka locally using Docker. A single-node development setup is useful for experimenting, but it does not demonstrate production fault tolerance.

Use a Kafka image and configuration compatible with the official Kafka distribution you choose. The following Compose example uses a KRaft-based single-node configuration, which does not require a separate ZooKeeper service.

Create a `compose.yaml` file:

```
services:
  kafka:
    image: apache/kafka:4.1.0
    hostname: kafka
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
```

Start Kafka:

```
docker compose up -d
docker compose ps
docker compose logs -f kafka
```

Important: This configuration is for local development. It uses plaintext communication, a single broker, and replication factors of one. Do not treat it as a production security or availability configuration.

If your chosen image version or distribution uses different environment-variable conventions, follow that image's documentation rather than mixing configuration formats.

12. Create a Topic

Run Kafka's topic tool inside the container:

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create \
  --topic orders \
  --partitions 3 \
  --replication-factor 1
```

List topics:

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --list
```

Describe the topic:

```
docker exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --topic orders
```

The description helps you inspect the partition count and replica assignment.

A single broker can only host one replica of each partition in this example, so a replication factor greater than one cannot be satisfied.

13. Produce Your First Events

Start the console producer:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders
```

Type a few lines:

```
{"eventId":"evt-1","eventType":"OrderCreated","orderId":"ord-100"}
{"eventId":"evt-2","eventType":"OrderCreated","orderId":"ord-101"}
{"eventId":"evt-3","eventType":"OrderCancelled","orderId":"ord-100"}
```

Each line is a separate record value.

In this simple console example, the producer does not automatically parse the JSON into structured fields. Kafka sees the record value as bytes; JSON interpretation is the application's responsibility.

Exit the producer with `Ctrl+C`.

14. Consume Your Events

Run a console consumer from the beginning of the retained stream:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --from-beginning
```

You should see the records you produced.

Now start a second consumer in a different terminal. It can also read the retained events. This illustrates that a consumer reading records does not normally remove them from the topic.

Stop both consumers with `Ctrl+C`.

Inspect consumer groups

Run a consumer with a group ID:

```
docker exec -it kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 \
  --topic orders \
  --group order-debug-group
```

After consuming some events, inspect the group:

```
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --list
```

Describe it:

```
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --group order-debug-group
```

Look at:

- Current offset
- Log-end offset
- Consumer lag
- Partition assignment, where shown by the tool

Consumer lag is a useful signal of how far behind a consumer group is from the end of the log. It does not, by itself, prove that the application is unhealthy. Interpret it alongside incoming throughput, processing latency, error rate, and business impact.

15. A Backend Example: Publishing an Order Event

Suppose you have a Spring Boot order service.

A simplified design might use a producer service:

```
@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventPublisher(
            KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public CompletableFuture<SendResult<String, OrderCreatedEvent>> publish(
            OrderCreatedEvent event) {

        return kafkaTemplate.send(
                "orders",
                event.orderId(),
                event
        );
    }
}
```

Illustrative event type:

```
public record OrderCreatedEvent(
        String eventId,
        String orderId,
        String customerId,
        BigDecimal totalAmount
) {}
```

Imports and configuration are omitted here. In a real Spring Boot application, configure the Kafka producer, JSON serialization, topic settings, and error handling for the versions you use.

Why use `orderId` as the key?

```
kafkaTemplate.send("orders", event.orderId(), event);
```

The key gives Kafka a way to choose a partition consistently under the configured partitioner. That can help preserve order for events belonging to the same order.

It does not guarantee that all orders are processed sequentially. Different orders can be processed concurrently.

What does the future mean?

The returned future allows the application to observe the result of the send operation. A successful producer send means Kafka acknowledged the record according to the configured producer and broker settings. It does **not** mean every downstream consumer has completed its work.

Avoid swallowing send failures. Handle them according to the API's business contract and the chosen reliability design.

The dual-write warning

Consider this implementation:

```
1. Save order in PostgreSQL.
2. Publish OrderCreated to Kafka.
```

What if step 1 succeeds but step 2 fails?

The order exists in the database, but downstream consumers may never hear about it.

Reversing the order introduces a different inconsistency: the event could be published even if the database transaction fails.

Do not assume two independent operations become atomic merely because they occur in the same method. Later engineering work will cover the transactional outbox pattern and related approaches.

16. Build a Consumer Responsibly

A consumer should not simply print a message and assume processing is complete.

For an order-processing consumer, define:

- What constitutes successful processing?
- Which failures should be retried?

- Which failures are permanent?
- When is offset progress committed?
- How are duplicate events handled?
- How is processing observed?
- What happens during shutdown or a rebalance?

A conceptual processing flow:

```
Receive record
    |
    v
Validate event
    |
    +---- Invalid permanently?
    |          |
    |          v
    |    Record failure / DLQ strategy
    |
    v
Perform idempotent business operation
    |
    +---- Temporary failure?
    |          |
    |          v
    |       Retry strategy
    |
    v
Commit progress according to the chosen strategy
```

This is a design outline, not a complete consumer implementation. Retrying synchronously can block a partition; retry topics, dead-letter topics, and pause/resume mechanisms have different ordering and operational tradeoffs.

17. Common Kafka Mistakes

Mistake 1: Treating offsets as global

Offset 10 in partition 0 is not the same record position as offset 10 in partition 1.

Mistake 2: Adding consumers without checking partition count

More consumers in one group do not guarantee more parallelism when the number of partitions is fixed.

Mistake 3: Assuming successful publication means successful processing

A producer acknowledgment and a consumer's business completion are different events.

Mistake 4: Assuming records disappear after consumption

Kafka generally retains records according to topic policy, not according to whether a particular consumer has read them.

Mistake 5: Assuming exactly-once behavior across every system

Kafka's transactional capabilities do not automatically make an external PostgreSQL transaction or payment request part of the same atomic operation.

Mistake 6: Using the wrong event key

If related events need ordering, an unsuitable key or partitioning strategy can undermine the intended processing model.

Mistake 7: Using a single local broker to claim high availability

A one-broker development setup cannot demonstrate broker redundancy or tolerate the loss of that broker.

Layer 3 — Production Engineering & Interview Mastery

18. Production Mental Models

Before operating Kafka in production, understand these relationships:

Throughput versus ordering: More partitions can enable more parallelism, but partition design and ordering requirements must be considered together.

Durability versus latency: Producer acknowledgments, replication, and in-sync replica settings affect the durability and availability tradeoffs. The word “Kafka” alone does not guarantee a particular durability level.

Delivery versus business effects: Kafka can redeliver records. Consumer operations must handle duplicates when the chosen processing model permits them.

Retention versus replay: Replay is possible only while the needed records remain available and compatible with the consumer's logic.

Consumer lag versus business health: Lag can indicate insufficient processing capacity, a stalled consumer, or a sudden increase in input volume. The number alone does not identify the root cause.

19. Foundational Incident Scenarios

Scenario A: Consumer lag keeps increasing

Potential causes:

- Incoming records are arriving faster than the consumer can process them.
- A consumer is stuck on slow database calls.
- The group has fewer effective consumers than expected.
- Errors or retries are consuming processing time.
- A partition is disproportionately hot.

Initial investigation:

1. Inspect consumer-group lag and per-partition distribution.

2. Compare input throughput with processing throughput.
3. Check application errors and processing latency.
4. Inspect downstream dependencies such as PostgreSQL or HTTP APIs.
5. Check partition assignments and recent rebalances.
6. Scale or optimize only after identifying the bottleneck.

Adding consumers may not help if the topic has too few partitions or the downstream database is already saturated.

Scenario B: Duplicate order processing

Potential causes:

- A consumer processed a record but crashed before committing its offset.
- The producer emitted the same logical event more than once.
- The application retried an operation after an ambiguous timeout.
- A replay intentionally reprocessed retained events.

Response:

1. Identify the stable event or business-operation ID.
2. Inspect the consumer's offset-commit strategy.
3. Make the business operation idempotent.
4. Consider database uniqueness constraints or an inbox/deduplication table.
5. Test crashes between business processing and offset commits.

Do not treat every duplicate as a Kafka defect. Duplicates can arise from normal retry and recovery behavior.

Scenario C: Events arrive in an unexpected order

Investigate:

- Whether the records use the intended key.
- Whether the relevant records are in the same partition.
- Whether the application processes records concurrently.
- Whether retry or dead-letter workflows reorder the effective business operations.
- Whether partition-count or partitioner changes affected placement.

Kafka's per-partition order cannot repair application-level reordering that occurs after records have been fetched.

Scenario D: Producer sends fail

Investigate:

- Broker connectivity and advertised listener configuration.
- Authentication and authorization, if enabled.
- Topic existence and metadata.
- Broker health and resource saturation.
- Serialization errors.
- Request timeouts and retry configuration.
- Replication and acknowledgment requirements.

A connection failure in a local Docker setup may be caused by using `localhost` from inside another container. Container-to-container clients typically need the Kafka service's network hostname and an advertised listener

appropriate to that network.

20. Kafka Interview Questions and Answers

Q1. What is Kafka?

Kafka is a distributed event-streaming platform that stores records in partitioned logs and allows independent consumers to process those records.

Q2. What is a topic?

A named stream of records divided into partitions.

Q3. What is a partition?

An ordered log within a topic. Partitions allow data distribution and parallel processing.

Q4. What is an offset?

A record's position within a particular partition. It is not globally unique across a topic.

Q5. What is a broker?

A Kafka server that stores and serves partitions.

Q6. What is a consumer group?

A group of consumers that cooperatively process assigned partitions. Different groups can independently consume the same topic.

Q7. Can two consumers in the same group process the same partition simultaneously?

Under the normal consumer-group assignment model, a partition is assigned to at most one group member at a time. Application-level concurrency can still affect the ordering of completed work.

Q8. What happens when a group has more consumers than partitions?

Some consumers remain without an assignment for that topic until assignments or membership change.

Q9. Does Kafka guarantee ordering?

Ordering is guaranteed within a partition, not globally across all partitions.

Q10. Why use a record key?

A key can influence partition selection and help keep related records in the same partition for ordered processing.

Q11. What is consumer lag?

A measure of how far a consumer group is behind the end of the log. Its exact interpretation depends on offsets and the monitoring method.

Q12. Does consuming a Kafka record delete it?

No, not ordinarily. Kafka retains records according to topic policies, independently of a particular consumer reading them.

Q13. What is the difference between Kafka and Redis?

Kafka focuses on distributed event streaming and partitioned logs. Redis is an in-memory data platform used for caching, fast data access, and other structures, including Streams. The right choice depends on the workload and required guarantees.

Q14. What is at-least-once processing?

A processing model in which a record may be processed more than once, so duplicates must be handled where they matter.

Q15. Does Kafka automatically guarantee exactly-once business operations?

No. Kafka offers transactional and idempotent features within specific boundaries, but external side effects require their own transactional or idempotency design.

Q16. Why can consumer lag increase?

Processing throughput may be lower than incoming throughput, a consumer may be stalled, retries may be excessive, or workload may be unevenly distributed.

Q17. What happens if a consumer crashes?

The group can reassign its partitions to remaining or replacement consumers. Records may be processed again depending on the committed offsets and processing strategy.

Q18. Why is a single-broker local setup insufficient for high availability?

There is no redundant broker to take over the lost broker's partition replicas. A local setup is useful for learning, not proof of production resilience.

Q19. Why is writing to a database and publishing to Kafka tricky?

The operations are usually not one atomic transaction. One can succeed while the other fails. Patterns such as transactional outbox and inbox/deduplication address different sides of this problem.

Q20. When should you not introduce Kafka?

Avoid it when the problem does not justify the operational and architectural complexity. A direct call, a database-backed job table, or a simpler queue may be more appropriate for a small workload or straightforward background task.

21. Hands-On Mastery Lab

Complete these tasks against your local Kafka environment.

- [ ] Start Kafka and verify the container is healthy.
- [ ] Create an `orders` topic with three partitions.
- [ ] Describe the topic and identify its partitions.
- [ ] Produce at least five sample order events.
- [ ] Consume the events from the beginning.
- [ ] Start a second independent consumer without sharing the same group ID.
- [ ] Run a consumer using a named group.
- [ ] Inspect the consumer group's offsets and lag.
- [ ] Explain why offset 2 in one partition is unrelated to offset 2 in another partition.
- [ ] Explain what happens when you add more consumers than the topic has partitions.
- [ ] Explain how using `orderId` as a key helps with ordering.
- [ ] Describe how a consumer crash could cause duplicate processing.
- [ ] Explain why a successful producer acknowledgment does not prove the inventory update succeeded.
- [ ] Identify one case where Redis would be a better fit and one case where Kafka would be a better fit.

Mastery challenge

Design a simple order-event architecture with:

- An Order API
- PostgreSQL
- A Kafka `orders` topic
- An inventory consumer
- An analytics consumer
- A Redis cache

Write down:

1. Which component owns the authoritative order state.
2. Which event is published when an order is created.
3. Which key should be used if order-level ordering is required.
4. How each consumer tracks progress.
5. What happens if the inventory consumer crashes after updating the database but before committing its offset.
6. How duplicate inventory updates can be prevented.
7. What the system should do if PostgreSQL saves the order but event publication fails.

Do not skip the final two questions. They are the bridge from Kafka fundamentals to real production engineering.

22. Chunk Completion Checklist

You have completed Kafka Chunk 1 when you can explain the following without memorizing definitions:

- [ ] Kafka as a distributed event log.
- [ ] Producer, broker, topic, partition, offset, and consumer.
- [ ] Consumer groups and partition assignment.
- [ ] Per-partition ordering.
- [ ] Retention and replay.
- [ ] Kafka versus traditional queues and Redis.
- [ ] At-most-once, at-least-once, and the boundaries of exactly-once processing.
- [ ] Basic local topic, producer, consumer, and consumer-group workflows.
- [ ] Why idempotency matters.
- [ ] Why database-plus-Kafka dual writes are difficult.
- [ ] How to start investigating consumer lag and duplicate processing.

Next: Kafka Chunk 2 — Producers, Consumers, Delivery Semantics & Spring Boot Integration

The next chapter will move from the mental model to implementation: producer configuration, consumer configuration, serialization, acknowledgment behavior, retries, offset commits, consumer groups, Spring Boot integration, and hands-on backend workflows.

Repository rule: Save this chapter as `content/kafka/01-kafka-fundamentals-event-streaming-core-mental-models.md`. Do not generate a chunk PDF. The final Kafka PDF is created only after all three Kafka chunks have been saved and verified.

PART II: PRODUCERS, CONSUMERS, DELIVERY SEMANTICS & SPRING BOOT INTEGRATION

Chunk 2 of 3 · Backend Engineering Handbook

Kafka — Chunk 2: Producers, Consumers, Delivery Semantics & Spring Boot Integration

Backend Engineering Tools & Infrastructure Handbook (BEH)

Source file: `content/kafka/02-kafka-producers-consumers-spring-boot-integration.md`

---

1. Learning Objectives

By the end of this chapter, you should be able to:

- Configure Kafka producers and consumers in a Spring Boot backend.
- Understand serialization, acknowledgments, retries, and idempotent producers.
- Explain consumer groups, partition assignments, and rebalancing.
- Manage offsets and understand the consequences of commit timing.
- Build practical event producers and consumers.
- Handle processing failures without creating uncontrolled retry loops.
- Design idempotent consumers and understand delivery guarantees.
- Diagnose common producer, consumer, and integration failures.

This chapter builds on Kafka's core mental models from Chunk 1. The focus is now implementation and operational behavior. The third chunk will deepen production reliability, security, performance, and incident response.

Layer 1 — Fundamentals & Mental Models

2. The Producer: Sending Records Reliably

A Kafka producer sends records to a topic. Its job is more involved than simply writing a message to a server.

A typical producer flow is:

```
Application creates event
        |
        v
Serialize key and value
        |
        v
Select partition
        |
        v
Buffer and batch records
        |
        v
Send records to broker
        |
        v
Receive acknowledgment or error
```

The producer must handle network failures, metadata updates, batching, timeouts, and potentially retriable errors.

2.1 Producer configuration that matters

Setting	Purpose	Engineering consideration
`bootstrap.servers`	Initial broker addresses	The client discovers cluster metadata through these addresses.
`key.serializer`	Serializes record keys	Must match the chosen key representation.
`value.serializer`	Serializes record values	JSON, strings, Avro, and other formats have different tradeoffs.
`acks`	Controls acknowledgment requirements	Affects durability and latency.
`enable.idempotence`	Prevents certain duplicate writes caused by producer retries	Does not make arbitrary business operations exactly once.
`retries`	Controls retry attempts in applicable configurations	Retries must be considered alongside timeouts and idempotency.
`delivery.timeout.ms`	Bounds the overall time allowed for delivery	Application timeout expectations should account for it.
`request.timeout.ms`	Bounds waiting for a broker response	Must be considered with retry and delivery settings.
`linger.ms`	Allows additional time to accumulate batches	Can improve batching at the cost of added waiting time.
`batch.size`	Controls the target batch size	The effect depends on traffic patterns and record sizes.
`compression.type`	Compresses producer batches	Can reduce network and storage usage while consuming CPU.

Configuration names and defaults can vary by Kafka client version. Check the documentation for the version used by your application.

2.2 Understanding `acks`

The `acks` setting determines what acknowledgment the producer requires before treating a send as successful.

- `acks=0`: the producer does not wait for broker acknowledgment.

- `acks=1`: the leader acknowledges the write.
- `acks=all`: the leader waits for the required in-sync replicas according to the cluster's replication configuration.

`acks=all` does not mean every configured replica must acknowledge under every configuration. The in-sync replica set and settings such as `min.insync.replicas` determine the relevant behavior.

A common reliability-oriented configuration uses:

```
acks=all
enable.idempotence=true
```

For stronger durability, the topic's replication configuration and broker health must also be designed appropriately. A single-broker development environment cannot demonstrate multi-broker fault tolerance.

2.3 Producer idempotence

Producer retries can create duplicates in some failure scenarios. Kafka's idempotent producer capability helps prevent duplicate log entries caused by supported producer retries within its protocol guarantees.

However, producer idempotence does not prevent your application from intentionally calling `send()` twice for the same business event.

For example:

```
Request arrives
    |
    v
Application creates OrderCreated
    |
    v
Application sends it twice
    |
    v
Two logical sends may occur
```

If the application must recognize the same business event across requests or replays, use a stable event ID and appropriate business-level deduplication.

3. Serialization: What Actually Goes Over the Network?

Kafka records contain byte-oriented keys and values. Producers serialize application objects before transmission, and consumers deserialize them after fetching records.

For a Java application:

```
OrderCreatedEvent object
        |
        v
JSON serializer
        |
        v
Kafka record bytes
        |
        v
JSON deserializer
        |
        v
OrderCreatedEvent object
```

Serialization is a contract between producers and consumers.

Common options include:

- Plain strings for simple events.
- JSON for approachable service integration.
- Avro or Protobuf when schema management and compact, typed data are important.

Schema evolution

Suppose a producer adds a new field:

```
{
  "eventId": "evt-123",
  "orderId": "ord-456",
  "currency": "INR"
}
```

Consumers must be able to interpret the event safely.

Good practices include:

- Prefer compatible schema changes.
- Define sensible behavior for missing optional fields.
- Avoid silently changing the meaning of an existing field.
- Test producer and consumer versions together.
- Use a schema registry when the architecture benefits from centrally governed schemas.

A schema registry is an additional component, not an inherent requirement for every Kafka deployment.

4. The Consumer: Fetching Is Not Processing

A consumer fetches records from partitions and executes application code.

Consider this sequence:

```
1. Fetch record at offset 25.
2. Update inventory in PostgreSQL.
3. Commit progress for the partition.
```

If the process crashes after step 2 but before step 3, the record may be processed again.

If the order is reversed:

```
1. Fetch record at offset 25.
2. Commit progress.
3. Update inventory in PostgreSQL.
```

A crash after step 2 but before step 3's business equivalent—the database update—can cause the operation to be skipped when the consumer resumes.

The core problem is that a Kafka offset commit and a PostgreSQL transaction are normally separate operations.

The solution is not simply to memorize a commit order. It is to design processing so retries and crashes are safe.

5. Offset Management

An offset records a consumer group's progress within a partition. The committed offset is used as a recovery position.

Important concepts:

- **Current position:** where the consumer is currently reading.
- **Committed offset:** the progress saved for recovery.
- **Log-end offset:** the end position of the partition log.
- **Consumer lag:** the gap between a consumer's progress and the log end, interpreted according to the monitoring tool's offset conventions.

Automatic versus manual commits

With automatic commits, the client periodically commits offsets according to configuration. This is convenient but can be unsafe if your processing takes longer than expected or if the commit timing does not match your business-processing requirements.

With manual commits, the application chooses when to commit. This provides more control but makes correct offset management the application's responsibility.

For business-critical processing, define the relationship between processing success and offset commits explicitly.

Example failure

```
Offset 25 fetched
       |
       v
Database update succeeds
       |
       v
Process crashes before offset commit
       |
       v
Offset 25 may be delivered again
```

This is a normal recovery scenario for an at-least-once processing design. Your application should be prepared for it.

6. Delivery Semantics in Practice

At-most-once

Progress may be committed before processing is complete. A crash can cause an event to be skipped from the application's perspective.

Use only when losing an event is acceptable under the system's requirements.

At-least-once

The application processes a record and commits progress after successful processing. A crash before the commit may cause duplicate processing.

This is a common approach for backend event processing when consumer operations are idempotent.

Exactly-once processing

Kafka transactions can coordinate certain Kafka reads, writes, and offsets. They are useful for pipelines in which consuming records and producing derived Kafka records must be handled transactionally.

They do not automatically include arbitrary database writes, HTTP calls, email sends, or payment-provider operations.

For external effects, design idempotency or a suitable cross-system consistency pattern.

Layer 2 — Practical Engineering & Workflows

7. Spring Boot Kafka Dependencies

For a Maven-based Spring Boot application, use Spring Kafka through the Spring Boot dependency-management system.

Example dependency:

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>
```

Use a Spring Boot-managed version unless you have a deliberate reason to override it. Keep Spring Boot, Spring Kafka, and the Kafka client versions compatible.

8. Configure the Application

A development configuration might look like this:

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: inventory-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        "[spring.json.trusted.packages]": "com.example.events"
```

This example assumes your event class is in `com.example.events` and that your Spring Kafka version supports these property names and behaviors. Confirm the trusted-package and deserialization configuration against your selected version.

For production:

- Store credentials outside source control.
- Configure authentication and encryption where required.
- Review serializer/deserializer security behavior.
- Define consumer commit and error-handling policies.
- Set resource and timeout limits based on actual workload.

`auto-offset-reset: earliest` applies when a group has no valid committed offset or its committed offset is no longer available. It does not mean the consumer always starts from the beginning.

9. Define the Event Type

```
package com.example.events;

import java.math.BigDecimal;

public record OrderCreatedEvent(
        String eventId,
        String orderId,
        String customerId,
        BigDecimal totalAmount
) {}
```

The event contains a stable identifier and business identifiers. A production event contract may also include a schema version, occurrence timestamp, trace context, and other fields required by the architecture.

Avoid placing internal secrets or unnecessary customer information in the event payload.

10. Implement a Producer

```
package com.example.orders;

import com.example.events.OrderCreatedEvent;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;

import java.util.concurrent.CompletableFuture;

@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate;

    public OrderEventPublisher(
            KafkaTemplate<String, OrderCreatedEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public CompletableFuture<SendResult<String, OrderCreatedEvent>> publish(
            OrderCreatedEvent event) {

        return kafkaTemplate.send(
                "orders",
                event.orderId(),
                event
        );
    }
}
```

Why use `orderId` as the key?

Related events for the same order can be routed to the same partition under the configured partitioning strategy, helping preserve their relative order.

What should the caller do?

The application must define how it reacts when publication fails.

For example, if the API responds with success after saving an order but event publication later fails, downstream systems may not learn about the order. If the API waits for publication, it still needs a clear failure policy and a solution for database/event consistency.

Do not hide publication failures with an empty exception handler.

11. Implement a Consumer

```
package com.example.inventory;

import com.example.events.OrderCreatedEvent;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class InventoryConsumer {

    @KafkaListener(
            topics = "orders",
            groupId = "inventory-service"
    )
    public void handle(OrderCreatedEvent event) {
        System.out.println(
                "Received order: " + event.orderId()
        );

        // Production code should invoke an idempotent
        // inventory-processing service here.
    }
}
```

This is a learning example. Printing an event is not the same as successfully updating inventory.

A real consumer should delegate to a business service that validates the event, performs an appropriate transaction, records idempotency state where necessary, and handles failures.

Consumer group design

Use a stable group ID for each logically independent processing application.

For example:

```
inventory-service  -> updates stock
analytics-service  -> records metrics
email-service      -> schedules notifications
```

If three replicas belong to `inventory-service`, they can share work through one group. The analytics service should generally have its own group if it needs to process every retained event independently.

12. Make the Consumer Idempotent

Suppose an inventory consumer receives the same `OrderCreated` event twice. A naive implementation might reserve stock twice.

One strategy is to record processed event IDs in the same database transaction as the business update.

Illustrative relational design:

```
CREATE TABLE processed_events (
    event_id VARCHAR(100) PRIMARY KEY,
    processed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Conceptual transaction:

```
BEGIN DATABASE TRANSACTION

Attempt to insert eventId into processed_events.

If eventId already exists:
    Treat this event as already processed.
Else:
    Apply inventory change.

COMMIT DATABASE TRANSACTION
```

The uniqueness constraint prevents two concurrent attempts from both successfully claiming the same event ID.

The deduplication record and business change must be coordinated in the same database transaction. If they are committed independently, a crash can still produce inconsistent outcomes.

This example also needs suitable handling for uniqueness violations, transaction retries, retention of deduplication records, and the specific business semantics of inventory reservation.

13. Handle Failures Deliberately

A consumer can fail for several reasons:

- Invalid event schema.
- Temporary database outage.
- Permanent business validation failure.
- Downstream timeout.
- Unexpected application exception.

These failures should not all be handled identically.

Failure	Possible strategy
Temporary dependency outage	Bounded retry with backoff
Invalid, unrecoverable payload	Record the failure and route to a dead-letter workflow
Duplicate event	Recognize it using idempotency state
Unexpected application bug	Preserve diagnostic context, alert, and avoid an infinite retry loop
Poison message	Isolate it so it does not indefinitely block useful progress

A retry strategy must account for partition ordering. Blocking a partition while retrying one record can delay later records in that partition. Moving failed records to another topic may improve progress but can alter the effective processing order.

Spring Kafka provides listener error-handling mechanisms, including error handlers and dead-letter publishing facilities. Configure them according to your version and ordering requirements rather than copying an unverified configuration blindly.

14. Local Integration Workflow

Use the local Kafka environment from Chunk 1.

1. Start Kafka using Docker Compose.

2. Create the `orders` topic.
3. Configure Spring Boot to connect to `localhost:9092`.
4. Start the application.
5. Publish an `OrderCreatedEvent`.
6. Verify the consumer receives the event.
7. Stop and restart the consumer.
8. Inspect the consumer group's offsets.
9. Simulate a temporary processing failure.
10. Verify that the event is retried or handled according to your configured policy.
11. Send the same event ID twice.
12. Verify that your idempotency mechanism prevents duplicate business effects.

For a multi-container setup, `localhost` inside the Spring Boot container refers to that container itself. Use the Kafka service hostname and the correct internal listener when the application also runs inside Docker Compose.

15. Debugging Checklist

When the producer appears to send but the consumer receives nothing, inspect the system systematically.

Producer

- Can it reach a bootstrap broker?
- Are advertised broker addresses reachable from the application?
- Is serialization succeeding?
- Are sends failing, timing out, or being retried?
- Is the target topic correct?

Topic and cluster

- Does the topic exist?
- Are the partitions available?
- Is the broker healthy?
- Are the topic's replication and in-sync replica settings compatible with the acknowledgment requirement?

Consumer

- Is it subscribed to the correct topic?
- Is the expected consumer group being used?
- Are committed offsets already at the end of the log?
- Is the deserializer failing?
- Is the consumer repeatedly failing while processing a record?
- Is the listener running and assigned partitions?

Networking

- Is the bootstrap address reachable?
- Are all advertised broker addresses reachable from the consumer?
- Is the client using the correct listener for its network location?
- Are authentication and TLS settings aligned?

Layer 3 — Production Engineering & Interview Mastery

16. Producer Reliability Tradeoffs

A producer configuration should match the business requirement.

For an important order event, a team may choose idempotent production, stronger acknowledgments, retries, and a carefully designed replication policy.

But configuration alone is not enough.

The application must also define what happens when:

- The send times out but the broker may have accepted the record.
- The application receives the same HTTP request twice.
- PostgreSQL commits but Kafka publication fails.
- Kafka publication succeeds but the HTTP response is lost.
- The caller retries after an ambiguous outcome.

These are distributed-systems failure modes, not merely Kafka configuration issues.

17. The Database–Kafka Dual-Write Problem

Consider:

```
BEGIN
  Insert order into PostgreSQL
COMMIT

Publish OrderCreated to Kafka
```

If the database commit succeeds but Kafka publication fails, the database and event stream disagree.

Now reverse the order:

```
Publish OrderCreated to Kafka

Insert order into PostgreSQL
```

If publication succeeds but the database transaction fails, consumers can observe an order that does not exist in the database.

Transactional outbox: the next architectural step

A common solution is the transactional outbox:

1. Save the business record in PostgreSQL.
2. Save an outbox event in the same database transaction.
3. A separate publisher reads unpublished outbox records.
4. The publisher sends events to Kafka.
5. Published records are marked or tracked for recovery.

Because the business record and outbox entry are written in one database transaction, the application avoids the original atomicity gap between those two writes.

The publisher can still send an event more than once if it crashes after publication but before recording success. Consumers must remain idempotent, or the publisher and downstream workflow must use an appropriate transactional design.

The outbox solves an important dual-write problem; it does not magically provide one global transaction across all services.

18. Consumer Rebalancing

A consumer group may rebalance when:

- A consumer joins.
- A consumer leaves or crashes.
- Subscription changes.
- Partition assignments need to be recalculated.

During a rebalance, partition ownership changes. Applications should handle revoked and assigned partitions correctly and avoid assuming that a consumer owns a partition forever.

Production implications:

- Long-running processing can delay progress.
- Poor shutdown handling can increase recovery time.
- Incorrect offset management can cause duplicate or skipped processing.
- Excessive membership changes can create repeated rebalances.

Useful operational signals include group membership changes, assignment changes, processing latency, lag, and repeated failures.

19. Security and Configuration Hygiene

For production deployments:

- Enable appropriate TLS and authentication.
- Use Kafka ACLs or equivalent authorization controls.
- Give applications only the permissions they need.
- Avoid committing credentials to source control.
- Restrict access to topics containing sensitive data.
- Review deserialization behavior and schema evolution.
- Monitor rejected requests and authentication failures.
- Separate development and production credentials and environments.

JSON deserialization of untrusted data should be configured carefully. Trusted-package settings are not a replacement for schema validation or authorization.

20. Production Incident Scenarios

Incident A: Producer latency increases

Investigate:

1. Producer send latency and error rate.
2. Broker request latency and resource saturation.
3. Network connectivity and DNS.
4. Batch size, compression, and `linger.ms`.
5. Retry rates and timeout configuration.
6. Replication health and in-sync replicas.

Do not immediately increase timeouts without determining why requests are slow.

Incident B: Consumer duplicates business operations

Investigate:

1. Whether duplicate event IDs are present.
2. Whether the consumer crashed after the business transaction but before offset commit.
3. Whether upstream producers intentionally or accidentally emitted duplicate logical events.
4. Whether retry behavior replays an operation.
5. Whether the business operation and deduplication record share a transaction.

Then fix the idempotency boundary and test the crash window.

Incident C: Consumer appears healthy but lag grows

Investigate:

- Incoming record rate versus processing rate.
- Processing time per event.
- Slow SQL or remote calls.
- Partition imbalance.
- Retry frequency.
- Consumer-group membership and assignments.

A running process is not proof that the consumer is making useful progress.

Incident D: One event blocks a partition

Investigate whether a poison message repeatedly fails. Determine whether it can be retried safely, isolated, or moved through a dead-letter workflow. Preserve the original payload, event identifiers, failure reason, and diagnostic context while avoiding sensitive-data exposure.

If strict per-key ordering matters, define how processing resumes after the failed event. Skipping it may violate business invariants.

21. Interview Questions and Answers

Q1. What does `acks=all` mean?

The leader waits for acknowledgment from the required in-sync replicas according to the cluster configuration. It does not universally mean every configured replica must acknowledge.

Q2. What is producer idempotence?

A Kafka producer feature that prevents certain duplicate writes caused by producer retries within its protocol guarantees. It does not prevent all duplicate business events.

Q3. What does a serializer do?

It converts an application's key or value into bytes for Kafka. A deserializer converts bytes back into application data.

Q4. Why is schema compatibility important?

Producers and consumers can be deployed independently. Incompatible changes can cause old consumers or new consumers to fail when reading events.

Q5. What is the difference between current position and committed offset?

The current position reflects where the consumer is reading. The committed offset is saved progress used for recovery.

Q6. Why can a consumer process a record twice?

It may finish the business operation and crash before committing progress. The record can then be delivered again.

Q7. Why is committing before processing risky?

A crash after the commit but before the business operation completes can cause the operation to be skipped on recovery.

Q8. Does manual offset committing guarantee exactly-once processing?

No. Manual commits provide control over progress timing, but external side effects can still be repeated or skipped unless the design handles failure windows.

Q9. How do you prevent duplicate database updates?

Use idempotent business logic and an appropriate deduplication mechanism, often enforced by a database uniqueness constraint in the same transaction as the business update.

Q10. What is a poison message?

A record that repeatedly fails processing, potentially preventing progress on its partition unless the application handles it.

Q11. What is a dead-letter topic?

A topic used to retain records that cannot be processed through the normal workflow, together with useful diagnostic context where appropriate.

Q12. What is a consumer rebalance?

A reassignment of topic partitions among members of a consumer group.

Q13. What does `auto-offset-reset=earliest` do?

It defines where to begin when the group has no valid committed offset. It does not override an existing valid committed offset.

Q14. What is the transactional outbox pattern?

A pattern in which the application writes business state and an outbox event in the same database transaction, then publishes the outbox event asynchronously.

Q15. Does a transactional outbox eliminate duplicates?

No. Publication can be repeated after a crash. Consumers should be idempotent or use an appropriate deduplication strategy.

Q16. Why might producer sends time out even if the record reached Kafka?

A response can be lost or delayed after the broker accepts a record. The producer may not know whether the write succeeded, which is why retry behavior and idempotence matter.

Q17. What is the difference between producer idempotence and a consumer inbox?

Producer idempotence addresses certain duplicate writes from producer retries. A consumer inbox or deduplication table records processed business event IDs to prevent repeated business effects.

Q18. How do you troubleshoot a consumer receiving no records?

Check connectivity, advertised listeners, topic subscription, group offsets, partition assignment, deserialization failures, and whether records are actually being produced.

Q19. Why can adding consumers fail to reduce lag?

The topic may not have enough partitions, the workload may be uneven, or a downstream dependency may be the bottleneck.

Q20. What does Kafka exactly-once semantics not automatically solve?

It does not automatically make arbitrary external database transactions, HTTP requests, payments, or emails exactly once.

22. Hands-On Assessment

Complete the following against your local Spring Boot project:

- [ ] Configure `KafkaTemplate` with a JSON serializer.
- [ ] Publish an `OrderCreatedEvent` using `orderId` as the key.
- [ ] Create a consumer using `@KafkaListener`.
- [ ] Run two instances in the same consumer group and observe partition assignment.
- [ ] Explain what happens when the group has fewer consumers than partitions.
- [ ] Explain what happens when the group has more consumers than partitions.
- [ ] Simulate a consumer crash before offset commit.
- [ ] Simulate a duplicate event ID.
- [ ] Implement an idempotency mechanism backed by PostgreSQL.
- [ ] Explain the difference between a producer acknowledgment and successful downstream processing.
- [ ] Draw the failure window between committing an order and publishing its event.
- [ ] Explain how a transactional outbox closes that particular dual-write gap.
- [ ] Explain why the outbox publisher and consumers must still account for retries.

23. Chunk Completion Checklist

You have mastered Chunk 2 when you can:

- [ ] Explain producer acknowledgment and idempotence.
- [ ] Choose appropriate serialization and schema-evolution practices.
- [ ] Configure a basic Spring Boot producer and consumer.
- [ ] Explain current position, committed offset, and lag.
- [ ] Reason about crashes before and after offset commits.
- [ ] Build an idempotent consumer.
- [ ] Explain consumer-group rebalancing.
- [ ] Design bounded retries and dead-letter handling.
- [ ] Diagnose common producer and consumer failures.
- [ ] Explain why the transactional outbox is useful.
- [ ] Distinguish Kafka-level guarantees from business-level guarantees.

Next: Kafka Chunk 3 — Production Reliability, Security, Performance & Incident Mastery

The final Kafka chapter will cover deeper production architecture, partition strategy, replication and durability, throughput tuning, lag diagnosis, consumer scaling, retries and dead-letter workflows, observability, security, failure simulations, and advanced interview scenarios.

Repository rule: Save this chapter as `content/kafka/02-kafka-producers-consumers-spring-boot-integration.md`. Do not generate a chunk PDF. The final Kafka PDF is created only after all three Kafka chapters have been saved and verified.

PART III: PRODUCTION RELIABILITY, SECURITY, PERFORMANCE & INCIDENT MASTERY

Chunk 3 of 3 · Backend Engineering Handbook

Kafka — Chunk 3: Production Reliability, Security, Performance & Incident Mastery

Backend Engineering Tools & Infrastructure Handbook (BEH)

Source file: `content/kafka/03-kafka-production-reliability-security-performance.md`

---

1. Learning Objectives

By the end of this chapter, you should be able to:

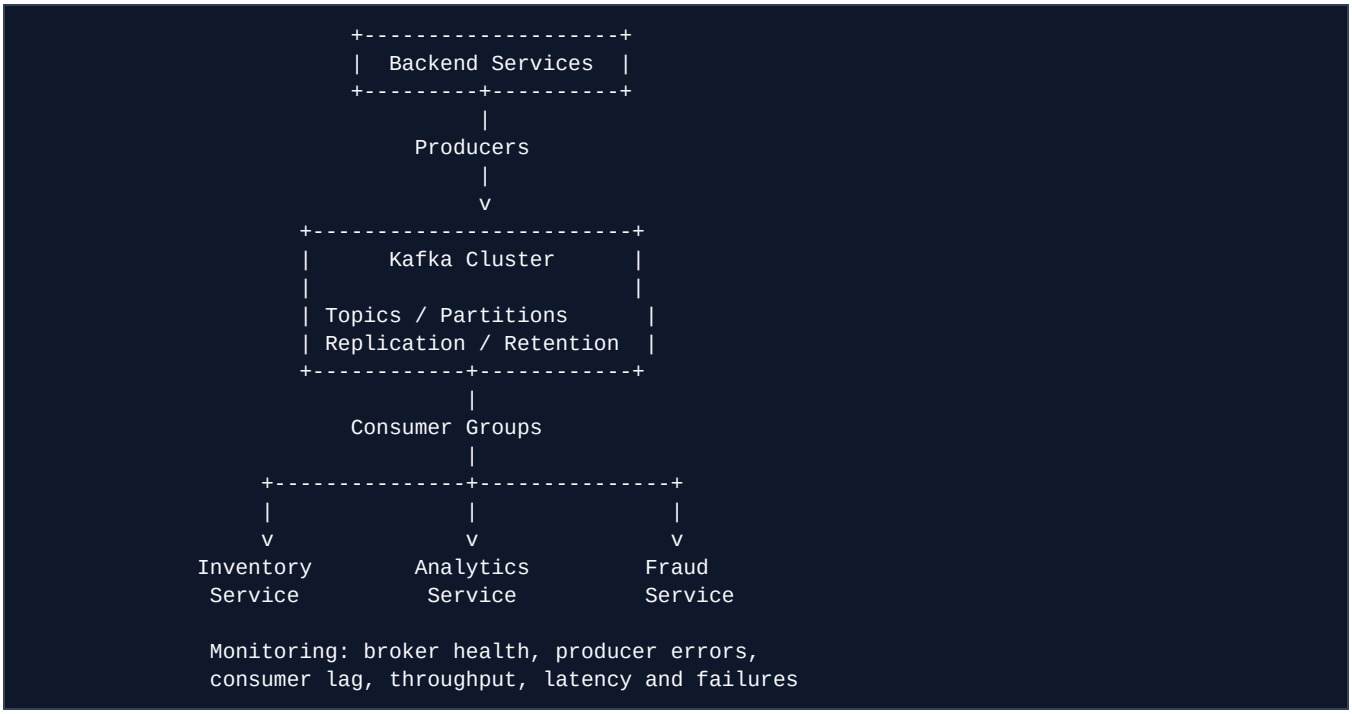
- Design partitioning strategies for backend workloads.
- Understand replication, in-sync replicas, and durability tradeoffs.
- Diagnose consumer lag and improve processing throughput.
- Configure retries, dead-letter workflows, and safe recovery procedures.
- Reason about Kafka outages, rebalances, and broker failures.
- Apply security, observability, and capacity-planning principles.
- Understand transactional processing boundaries and the outbox pattern.
- Troubleshoot production incidents systematically.
- Defend Kafka architecture decisions in SDE-1 and backend engineering interviews.

This is the final Kafka chapter. Across the three planned chunks, the Kafka curriculum covers fundamentals, practical implementation, and production engineering.

Layer 1 — Fundamentals & Mental Models

2. Kafka in Production: The System, Not Just the Broker

A production Kafka deployment is more than a collection of brokers. It includes producers, consumers, topic configuration, replication, authentication, monitoring, and operational procedures.



The engineering objective is not simply to keep Kafka running. It is to ensure the overall event-processing system meets its required correctness, latency, availability, and recovery goals.

A healthy broker can coexist with an unhealthy application. For example, the cluster may accept records successfully while a consumer is stuck waiting for a database connection.

3. Partition Strategy and Parallelism

Partitions are the foundation of Kafka's parallelism.

Suppose the `orders` topic has six partitions and the `inventory-service` consumer group has three consumers.



The actual assignment depends on the group assignor and current membership. The diagram illustrates one possible assignment.

3.1 More partitions, more potential parallelism

For a single consumer group, a partition can be assigned to at most one group member at a time under the normal group-assignment model.

If there are six partitions and three active consumers, all three consumers can process assigned partitions concurrently.

If the group grows to ten consumers, four consumers may have no partition assigned for that topic.

Adding partitions can enable more consumer parallelism, but it is not a free optimization.

Consider:

- Broker storage and metadata overhead.
- File handles and memory overhead.
- Consumer assignment and rebalancing cost.
- The ability of downstream databases to handle additional concurrency.
- Ordering requirements.
- Future workload growth.

Do not choose a partition count solely because a large number sounds scalable.

3.2 Choosing a partition key

For an order-processing workload, a key might be:

```
orderId
```

For a customer-account workload, a key might be:

```
accountId
```

The key should reflect the entity whose events need related placement or ordered processing.

A poor key can create a **hot partition**.

Imagine one customer generates 40% of all traffic. If the customer ID is the key, a disproportionate share of traffic may go to one partition.

This limits the parallelism available for that workload.

Potential responses include:

- Reconsidering the partition key.
- Separating unusually hot workloads.
- Increasing partitions where it genuinely helps.
- Optimizing consumer processing.
- Using a different data model if strict ordering is not essential.

Changing the key may change ordering guarantees. It is a business and architecture decision, not merely a tuning adjustment.

4. Replication, Leaders, and In-Sync Replicas

Kafka can replicate partitions across brokers.

A partition has a leader replica that handles normal reads and writes, while follower replicas replicate its log.

```
Partition 0

Broker A: Leader
    |
    +------> Broker B: Follower
    |
    +------> Broker C: Follower
```

This is an illustrative replica arrangement. Real assignments depend on cluster metadata and configuration.

4.1 Why replication matters

If a broker fails, the cluster may be able to elect a new leader from an eligible replica. That is how replication supports availability and recovery.

Replication does not guarantee that every failure is harmless. The outcome depends on:

- Which replicas are available.
- Whether replicas are in sync.
- The producer's acknowledgment requirements.
- The topic's replication factor.
- The minimum in-sync replica configuration.
- Cluster health and controller behavior.

4.2 `acks=all` and `min.insync.replicas`

These settings work together.

- `acks=all` asks the leader to wait for the required in-sync replica acknowledgments.
- `min.insync.replicas` establishes a minimum in-sync replica count required for writes under the relevant acknowledgment behavior.

For example, a topic with replication factor three and a minimum in-sync replica count of two can reject writes requiring all acknowledgments when fewer than two eligible in-sync replicas remain.

That rejection is intentional: it can prevent accepting writes under conditions that violate the desired durability policy.

The correct configuration depends on the application's tolerance for data loss and temporary write unavailability.

4.3 Replication is not a backup

Replication helps with broker failure, but it does not protect against every logical error.

If an application writes incorrect data, that data may be replicated too. Retention or deletion mistakes can also affect the records you need.

A robust data-protection strategy considers backups, recovery procedures, replication, and application-level safeguards separately.

5. Retention, Compaction, and Storage

Kafka stores records according to topic-level policies.

Two important retention concepts are:

Time/size-based retention: Records become eligible for deletion according to the configured retention policy.

Log compaction: Kafka retains the latest value for each key in a compacted log, subject to compaction behavior and configuration. Compaction is useful when consumers need the latest known state for keys rather than every historical transition.

Example state stream:

```
customer-1 -> Bronze
customer-1 -> Silver
customer-1 -> Gold
```

A compacted topic can eventually retain the latest value for `customer-1`, rather than preserving every superseded value indefinitely.

This is not the same as a complete event-history log.

Choose retention based on replay requirements, storage costs, compliance obligations, and the downstream systems' recovery needs.

6. Consumer Lag: The Core Operational Signal

Consumer lag indicates how far a consumer group is behind the end of the partition log, according to the offsets used by the monitoring tool.

Conceptually:

```
Lag ≈ Log-End Position - Consumer Progress
```

The exact numeric calculation can differ according to offset conventions and tooling.

Lag is not automatically a failure. A short-lived spike during a traffic burst may be expected. A steadily growing backlog that misses business deadlines is more concerning.

Understand lag alongside throughput

Suppose an application receives 2,000 records per second but processes only 1,500 records per second.

Its backlog grows by approximately 500 records per second, assuming a sustained period and comparable units.

Adding consumers may help if the workload can be parallelized and the downstream dependencies have capacity. It will not fix a database that is already saturated.

7. Delivery Guarantees and Transaction Boundaries

Kafka's reliability mechanisms operate within defined boundaries.

Producer reliability

Producer idempotence helps prevent certain duplicate writes caused by retries.

Kafka transactions

Transactions can coordinate supported Kafka writes and offsets. They are particularly useful when one Kafka processing stage consumes records and publishes derived records.

External side effects

PostgreSQL transactions, HTTP calls, email delivery, and payment-provider operations are not automatically part of a Kafka transaction.

For these systems, use application-level patterns such as:

- Idempotent operations.
- Database uniqueness constraints.
- Transactional outbox.
- Inbox or deduplication tables.
- Explicit retry and reconciliation workflows.

A production architecture should state its guarantee precisely rather than simply claiming “exactly once.”

Layer 2 — Practical Engineering & Workflows

8. Diagnose Consumer Lag Systematically

When lag grows, investigate before scaling.

Step 1: Inspect the consumer group

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe \
  --group inventory-service
```

Review the lag per partition, current consumer progress, and assignment information provided by the tool.

Step 2: Compare incoming and processing throughput

Measure:

- Records produced per second.
- Records processed per second.
- Average and tail processing latency.
- Error and retry rates.
- Lag growth over time.

Step 3: Inspect the downstream dependency

If each event triggers a database update, inspect:

- Database query latency.
- Connection-pool saturation.

- Lock contention.
- Database CPU and I/O.
- Transaction retries.
- Timeouts.

Step 4: Inspect partition distribution

A single partition may be lagging because one key dominates traffic or because a specific partition's processing is blocked.

Step 5: Choose the intervention

Finding	Potential intervention
Consumer logic is CPU-bound	Profile and optimize processing
Database calls are slow	Optimize SQL, indexes, batching, or pool usage
Too few partitions for useful parallelism	Evaluate a partition expansion or topic redesign
One hot key dominates	Revisit key strategy or isolate the workload
Poison messages cause repeated failure	Introduce bounded retries and a dead-letter strategy
Consumers repeatedly restart	Investigate crashes, memory pressure, timeouts, or deployment instability

Do not increase concurrency blindly. More consumers can overwhelm downstream services and make the incident worse.

9. Throughput Tuning

Kafka throughput depends on the entire pipeline: producer batching, network, broker resources, replication, partition count, consumer processing, and downstream dependencies.

9.1 Producer batching

Relevant settings include:

```
linger.ms=5
batch.size=32768
compression.type=snappy
```

These values are examples, not universal recommendations.

- `linger.ms` can allow additional records to accumulate into a batch.
- `batch.size` influences the producer's target batch size.
- Compression can reduce network and storage usage at the cost of CPU.

Benchmark with realistic payload sizes and traffic patterns.

9.2 Consumer processing

A consumer's throughput can be limited by:

- CPU-intensive processing.
- Synchronous network calls.
- Database transactions.
- Serialization overhead.
- Excessive logging.
- Retry delays.
- Too much contention between worker threads.

Measure the slowest part of the pipeline instead of assuming Kafka itself is the bottleneck.

9.3 Batching database writes

If the business logic permits it, batching multiple records into a database operation can improve throughput.

But batching introduces tradeoffs:

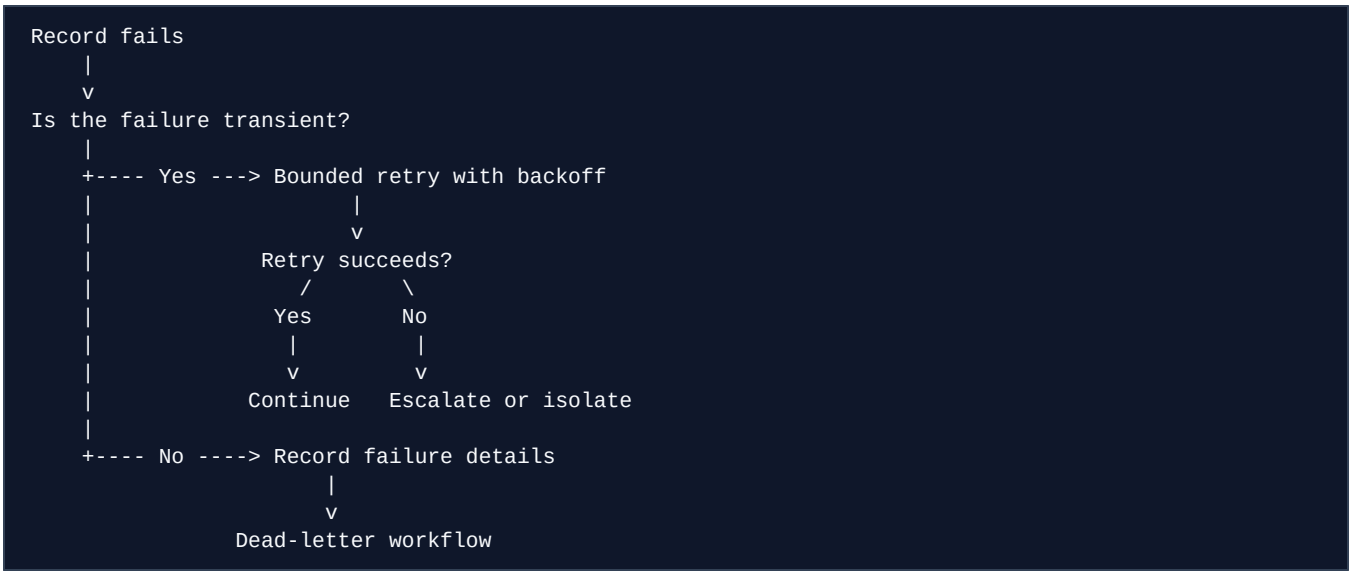
- Higher latency before a batch is complete.
- More complex error handling for partially invalid data.
- Larger transaction scope.
- More complicated offset-commit decisions.

The batch's business operations and offset progress must be coordinated so that failed work is not silently skipped.

10. Retries and Dead-Letter Workflows

Not every error should be retried indefinitely.

A practical classification is:



A dead-letter topic can preserve a record that could not be processed by the normal workflow.

Useful diagnostic information can include:

- Original topic, partition, and offset.
- Event ID.
- Failure category.

- Timestamp and retry count.
- A sanitized error description.
- Trace or correlation ID.

Do not blindly copy secrets or sensitive payload fields into diagnostic records.

Ordering tradeoff

Moving a failed event to a dead-letter topic and continuing with later records can change the effective order in which business effects occur.

If order matters, define whether later records must wait, whether the entity can be quarantined, or whether the failure can be resolved and replayed safely.

11. Safe Replay and Recovery

Kafka allows consumers to reread retained records by changing the group's starting position or using a separate consumer group.

Replay is useful for:

- Rebuilding derived state.
- Reprocessing after a fixed bug.
- Backfilling analytics.
- Recovering a consumer's application state.

But replaying events can repeat external side effects.

Before replaying production data:

1. Identify the exact topic, partitions, and offset range.
2. Confirm retention still includes the required records.
3. Determine whether the operation is idempotent.
4. Decide whether to reuse the existing group or create a separate recovery group.
5. Estimate replay volume and downstream load.
6. Define a stop condition and rollback or reconciliation plan.
7. Monitor lag, processing errors, and business-level outcomes.

Avoid changing production offsets casually. A mistaken offset reset can cause missed processing or large-scale duplication.

12. Security and Access Control

A production Kafka deployment should apply defense in depth.

Authentication

Verify the identity of clients using the chosen mechanism, such as SASL-based authentication or another supported enterprise configuration.

Encryption

Use TLS where required to protect traffic in transit.

Authorization

Restrict access to topics, consumer groups, and administrative operations.

For example, an analytics consumer may need permission to read the `orders` topic but should not automatically receive permission to delete topics or modify cluster-wide settings.

Secret management

- Keep passwords, tokens, and private keys out of source control.
- Use a secret manager or controlled deployment configuration.
- Rotate credentials.
- Audit access.
- Restrict administrative access.
- Avoid exposing event payloads in logs unnecessarily.

Security configuration depends on the Kafka distribution, deployment model, and selected authentication and authorization mechanisms.

13. Observability: What Should You Monitor?

Monitor both Kafka infrastructure and application behavior across four essential dimensions:

Monitoring Area	Focus Metrics & Operational Signals
Broker health	Broker availability, request latency, resource pressure, under-replicated partitions, and offline partitions.
Producer health	Send failures, retries, request latency, throughput, and serialization errors.
Consumer health	Lag, processing throughput, processing latency, failed records, retries, and rebalances.
Business outcomes	Orders processed, inventory updates completed, duplicate-event detections, and end-to-end processing delay.

Infrastructure metrics tell you whether Kafka is functioning. Business metrics tell you whether the system is accomplishing its purpose.

For example, a low consumer lag is not enough if the consumer acknowledges records but fails to update inventory correctly.

14. End-to-End Tracing

Distributed event processing can cross multiple services and execute long after the original HTTP request ends.

Use correlation identifiers and distributed tracing to connect:

```
HTTP request
    |
    v
Order API
    |
    v
Kafka publish
    |
    v
Inventory consumer
    |
    v
PostgreSQL transaction
```

Where supported, propagate trace context through Kafka headers and instrument producer and consumer operations.

A trace ID is useful for investigation, but it should not be treated as a unique business-event identifier. Use a separate stable event ID for deduplication.

15. Production Readiness Checklist

Before deploying an event-driven service, review the following.

- [ ] Topic replication and minimum in-sync replica settings match durability requirements.
- [ ] Producer acknowledgment, retry, and idempotence settings have been reviewed.
- [ ] Consumer offset-commit behavior is explicit.
- [ ] Business operations are safe under duplicate delivery.
- [ ] Schema evolution is tested.
- [ ] Retry and dead-letter behavior is defined.
- [ ] Consumer lag and processing latency are monitored.
- [ ] Broker and consumer failure alerts are actionable.
- [ ] Authentication, encryption, and authorization are configured as required.
- [ ] Secrets are managed outside source control.
- [ ] Replay and offset-reset procedures are documented.
- [ ] Downstream database capacity has been tested.
- [ ] Graceful shutdown and rebalance behavior have been tested.
- [ ] Disaster recovery and data-protection requirements are documented.
- [ ] Production load testing uses realistic event sizes and arrival rates.

Layer 3 — Production Engineering & Interview Mastery

16. Incident Simulation 1: Consumer Lag Explosion

Situation: The `orders` topic is receiving traffic normally, but inventory consumer lag has grown rapidly.

Investigation

1. Check the lag by partition.
2. Compare incoming rate with processing rate.
3. Check recent deployments and error-rate changes.
4. Inspect database latency and connection-pool saturation.
5. Look for repeated retries or a poison event.
6. Determine whether one partition is disproportionately affected.

Decision

If processing is slow because database queries are inefficient, optimize the database path first.

If processing is healthy but there are too few partitions for the required concurrency, evaluate partition scaling.

If a poison event is blocking progress, resolve or isolate it according to the ordering requirements.

Verification

Confirm that lag is decreasing, error rates are stable, downstream services remain healthy, and the business processing delay is returning to its target.

Do not declare recovery merely because consumer processes are running.

17. Incident Simulation 2: Duplicate Inventory Reservations

Situation: A customer places one order, but inventory appears to have been reserved twice.

Possible causes

- The consumer updated the database and crashed before committing its offset.
- The producer emitted duplicate logical events.
- An upstream request was retried.
- A replay operation repeated the same event.
- The consumer's deduplication logic is incorrect.

Investigation

1. Compare the event IDs and order IDs.
2. Inspect consumer restart and offset-commit timing.
3. Check application retry logs.
4. Examine the inventory transaction history.
5. Verify whether event deduplication and the business update share a transaction.

Resolution

Implement or repair idempotent processing, reconcile incorrect inventory state, and add a regression test for the crash window.

A robust test should simulate failure after the database transaction commits but before Kafka progress is committed.

18. Incident Simulation 3: Producer Errors After a Broker Failure

Situation: A broker fails and order-event publication begins returning errors.

Investigation

1. Confirm the failed broker and affected partitions.
2. Inspect leader elections and replica health.
3. Check the in-sync replica count.
4. Verify producer error and retry metrics.
5. Confirm that clients can reach the advertised broker addresses.
6. Determine whether writes are being rejected because durability requirements cannot be met.

Resolution principles

- Preserve the intended durability policy.
- Restore broker and replica health.
- Allow safe retries according to producer configuration.
- Avoid disabling acknowledgment requirements merely to hide errors.
- Verify event completeness and business reconciliation after recovery.

The correct response depends on the cluster topology and configured guarantees. A single-broker local environment cannot simulate a genuine replicated-cluster failover.

19. Incident Simulation 4: Poison Event Blocks Business Progress

Situation: One malformed event repeatedly fails and later records in the partition are not being processed.

Investigation

- Identify the record's topic, partition, offset, and event ID.
- Determine whether the failure is transient or permanent.
- Check whether the consumer retries indefinitely.
- Establish whether subsequent events depend on the failed event.

Resolution

Use a bounded retry strategy and an appropriate isolation workflow.

If later records are allowed to proceed, ensure that skipping or delaying the failed event does not violate business invariants. Preserve enough diagnostic information to repair and replay the event if necessary.

20. Advanced Interview Questions

Q1. How would you choose the number of partitions?

Estimate required throughput and parallelism, consider consumer count and key distribution, test realistic workloads, and account for operational overhead and ordering requirements. Avoid choosing an arbitrary large number.

Q2. What happens when a broker fails?

The cluster may elect a new leader from an eligible replica. The outcome depends on replica availability, in-sync status, cluster metadata, and durability configuration.

Q3. Does replication prevent data loss?

It reduces exposure to certain broker failures but does not eliminate every data-loss scenario. Producer acknowledgment policy, replica health, retention, and logical errors all matter.

Q4. Why can `acks=all` still fail to protect a write?

It depends on the configured in-sync replica set and cluster state. If the required acknowledgment conditions cannot be met, the write may fail. The setting does not replace appropriate replication and recovery design.

Q5. Why does consumer lag increase even though Kafka brokers are healthy?

Consumers may be slower than incoming traffic, blocked by downstream dependencies, retrying failed records, or processing unevenly distributed partitions.

Q6. How would you scale a slow consumer?

Identify the bottleneck first. Then consider optimizing processing, batching, increasing consumer concurrency within partition limits, or changing partition strategy. Check downstream capacity before scaling.

Q7. What is a hot partition?

A partition receiving a disproportionate share of traffic or expensive work, often because of key distribution or workload characteristics.

Q8. Why is changing a partition key dangerous?

It can change record placement and the ordering relationships the application relies on. Existing and future events may no longer share the same partitioning behavior.

Q9. What is the difference between retention and compaction?

Retention removes records according to configured policies. Compaction retains the latest known values for keys over time, subject to its configuration and processing behavior.

Q10. How does a dead-letter topic help?

It isolates records that cannot be processed normally, enabling investigation and recovery without necessarily allowing one poison record to block all further processing.

Q11. What is the downside of dead-lettering a failed record?

Continuing with later records can change effective business order. The workflow must account for dependencies and define how failed records are recovered.

Q12. How do you safely replay production events?

Define the topic and offset range, check retention, assess side effects and idempotency, control replay rate, monitor downstream load, and define reconciliation and stop conditions.

Q13. What is the difference between an event ID and a trace ID?

An event ID identifies a logical event for deduplication and business tracking. A trace ID links operations across a distributed execution. One event may be processed across multiple spans, and tracing identifiers should not replace business identity.

Q14. What should you monitor beyond consumer lag?

Processing latency, throughput, error and retry rates, broker health, replication status, producer failures, downstream latency, and business completion metrics.

Q15. Why is a Kafka transaction not a global transaction?

Kafka transactions coordinate supported Kafka operations. They do not automatically include arbitrary databases or external HTTP services.

Q16. What problem does the transactional outbox solve?

It coordinates the durable creation of business state and an event record in the same database transaction, reducing the risk of a committed business change with no corresponding event record.

Q17. Does an outbox eliminate duplicate publication?

No. A publisher may send an event and crash before recording publication success. Consumers still need idempotency or an equivalent deduplication strategy.

Q18. What is the role of consumer-group rebalancing?

It redistributes partition ownership as membership or assignment requirements change. Poor processing and shutdown behavior can make rebalances disruptive.

Q19. Why is blindly increasing retries dangerous?

It can increase latency, consume resources, block progress, and repeatedly hit a dependency that is already failing. Retries should be bounded and appropriate to the failure type.

Q20. When is Kafka unnecessary?

When the workload is simple, throughput and replay requirements are modest, and a direct call, database-backed job, or simpler queue satisfies the reliability requirements with less operational complexity.

21. Final Hands-On Capstone

Build an event-driven order-processing prototype using Spring Boot, PostgreSQL, Kafka, and Redis.

Architecture

```
Client
  |
  v
Order API
  |
  +------> PostgreSQL
  |          |
  |          +----> Orders table
  |          |
  |          +----> Outbox table
  |
  v
Outbox Publisher
  |
  v
Kafka: orders topic
  |
  +------> Inventory Consumer
  |
  +------> Analytics Consumer

Redis: cache frequently accessed order summaries
```

This is a reference architecture. Redis is optional and should be included only if the caching requirement justifies it.

Implementation milestones

Milestone 1 — Produce events

- Create an order through a REST endpoint.
- Persist the order in PostgreSQL.
- Define an `OrderCreated` event.
- Publish through Kafka.

Milestone 2 — Process events

- Build an inventory consumer.
- Build an independent analytics consumer.
- Use separate consumer groups.
- Add stable event IDs and structured logging.

Milestone 3 — Reliability

- Add a PostgreSQL-backed idempotency mechanism.
- Implement an outbox if order persistence and event publication must be reliably coordinated.
- Add bounded retries and a dead-letter workflow.
- Define how failed events are recovered.

Milestone 4 — Observability

- Measure producer send errors and latency.

- Monitor consumer lag and processing time.
- Log event IDs and correlation IDs.
- Track business-level completion counts.

Milestone 5 — Failure testing

Simulate:

- A consumer restart after processing but before committing progress.
- A duplicate event.
- A database outage.
- A malformed event.
- A slow consumer and growing lag.
- A publication failure.
- A replay of retained events.

For each scenario, document expected behavior, observed behavior, recovery procedure, and the test proving recovery.

22. Final Kafka Mastery Assessment

Review and validate the following Kafka readiness checklist before production deployment:

- [ ] Explain topics, partitions, offsets, brokers, producers, and consumer groups.
- [ ] Design a key strategy that supports the required ordering guarantees.
- [ ] Explain producer acknowledgments, idempotence, and replication.
- [ ] Configure a Spring Boot producer and consumer.
- [ ] Explain offset commits and duplicate-processing failure windows.
- [ ] Implement an idempotent consumer backed by PostgreSQL.
- [ ] Explain the transactional outbox and its remaining duplicate-publication risk.
- [ ] Diagnose lag using throughput, latency, partition distribution, and downstream health.
- [ ] Design bounded retries and dead-letter recovery.
- [ ] Explain retention, compaction, and safe replay.
- [ ] Identify Kafka security and observability requirements.
- [ ] Defend a Kafka architecture using concrete reliability and operational tradeoffs.

> **Mastery Validation:** Validate all checklist items through hands-on testing, the capstone architecture, and failure simulations before considering the material fully mastered.

23. Kafka Curriculum Completion

The three planned Kafka chapters are now authored in this conversation:

1. Fundamentals, Event Streaming & Core Mental Models
2. Producers, Consumers, Delivery Semantics & Spring Boot Integration
3. Production Reliability, Security, Performance & Incident Mastery

Repository completion is a separate step. Save all three Markdown sources under `content/kafka/`, review the examples against the Kafka and Spring Kafka versions you use, and verify the files before generating the single final PDF:

`docs/kafka/BEH_KAFKA_COMPLETE_HANDBOOK.pdf`

Do not create PDFs per chunk. Once the complete Kafka handbook PDF is generated and verified, use the required Git commit message:

`Kafka done`

The next tool in the locked curriculum is **Nginx — Chunk 1 of 2: Reverse Proxies, HTTP Routing & Core Mental Models**.